# FLASH MDP - Consumer Sentiment Analysis


Luyang Hu
hulu@umich.edu

Jinyi Wang
jinyi@umich.edu

Zhe Chen
czhe@umich.edu

Lei Lin
leilin@umich.edu

Arjun Patel
patelarj@umich.edu

Rui Wu
ruiwu@umich.edu

Kayvan Najarian *UM Faculty Mentor*
kayvan@med.umich.edu

Hunter Dunbar *FLASH Sponsor*
hunter.dunbar@flashparking.com

Edward Hunter *FLASH Sponsor*
edward.hunter@flashparking.com


## Important Note

This PDF file is a collection of our documentation in private GitHub Repo, which has been sent to both our faculty mentor and industry sponsor. It's important to note that this document should only serve as a reference. Information on the GitHub repository is more detailed and is a better demonstration of our complete work. We recommend referring to both when accessing our work. Thank you!

## Project Video


Figure 1: Code Demo

## Project Description

With no consistent metric for FLASH Parking to measure customer satisfaction, the 2023 FLASH MDP Cohort is tasked with developing an automated system to measure customer satisfaction. This system will utilize visual and audio data from parking kiosks. The project is divided into 3 major parts:

- The Object Detection Subteam focuses on identifying human faces and extracting relevant audio snippets from video footage.
- The Visual Sentiment Subteam focuses on classifying sentiment from images.
- Audio Sentiment Subteam focuses on classifying sentiment from audio.

We are responsible for developing, training, and testing various customer satisfaction assessment models. These models are integrated into automated software for efficient data processing and insight generation.
In this github repository, we include both the origin code and detailed description of the current works as well as some legacy models. Ultimately, the pipeline will deliver a sentiment tag (Positive or Negative) and a multimodal sentiment score range [0, 1] by analyzing the input. The user input could be in formats of (1) a self-recorded video clip or (2) the real-time camera recording. Detail user manual see sections below.

## System Diagram

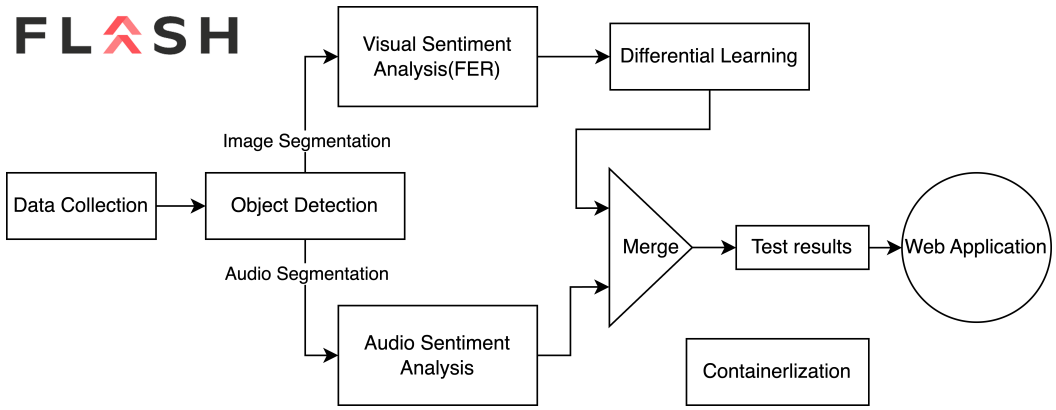


Figure 2: System Diagram

## Prerequisites

It is always a good practice to create a virtual environment to avoid conflicts between different projects. To do so, run the following command:

```
python -m venv env # create a virtual environment under directory env
source env/bin/activate # activate the virtual environment
```

Note: please check your python version and path before running these commands.
Before running the script, ensure you have installed all necessary Python libraries, all libraries are stated in 'requirements.txt', simply run:

```
pip install -r requirements.txt
```

Note: There could be some packages that need to be installed manually. Situations vary for different devices. Then you need to manually download two folders. We are not able to write scripts to download private files.

1. Download the `dataset` folder.
2. Download the `saved_models` folder.

Check the repository file tree to make sure everything is included.

### Repo File tree

```
.
 README.md
 dataset                          # Download from link, includes all training and testing datasets
    LSTM_training                # LSTM training dataset
    ViT_training                 # ViT training dataset
    random_data                  # Random images for testing purpose
    self-recorded_testing        # Self-recording dataset, private use only
 output                           # Store segmented driver faces and audio
 requirements.txt                 # Virtual Environment requirements
 result                           # Store differential learning graphs for each analysis
 saved_models                     # Saved model parameters, used to load trained model in analysis phase
    audio_model
    object_detect_model
    visual_model
 scripts                          # Running scripts regarding the pipeline
 src                              # Contain all source code
```

## Usage

We created a bunch of bash scripts for the ease of running our pipeline. First, make the script executable by:

```
cd scripts
chmod +x run.sh
chmod +x empty.sh
```

Run using the following command in the root directory:

```
./scripts/run.sh
```

`<video_file_path>` (optional): Video source. Default is 0 (camera). You can specify a video file or a different camera source.
The script will display the video stream with face detection. Audio recording will be triggered when a face is detected, and the recorded audio will be saved as a WAV file. Press Ctrl+C to exit the script, and the pipeline will jump to sentiment analysis algorithms.
Note that the path needs to be relative to the root `dataset/self-recorded_testing/01.mov`
If you would like to empty the `output` folder, use:

```
./scripts/empty.sh
```

## Object Detection(Lei Lin & Arjun Patel)

This Python script demonstrates an application that uses the YOLOv8 model for real-time object tracking in video frames captured from a webcam. It also includes functionality for audio recording when an object is detected, saving the recorded audio as a WAV file.

### Dataset

To train our YOLO model, we created a dataset by adapting existing datasets to better fit our model environments. We selected images from the "Hands-on-Wheel" databases because it contained images of drivers inside of a car, similar to the view expected from the kiosk's POV looking at cars.
Using Roboflow, we removed the labels from the "Hands on Wheel" database and manually created bounding boxes for the drivers' faces. The object detection subteam created over 3500 self-annotated images for training. The dataset can be found here: Facial Detection Dataset.

# Model

## Overall Structure

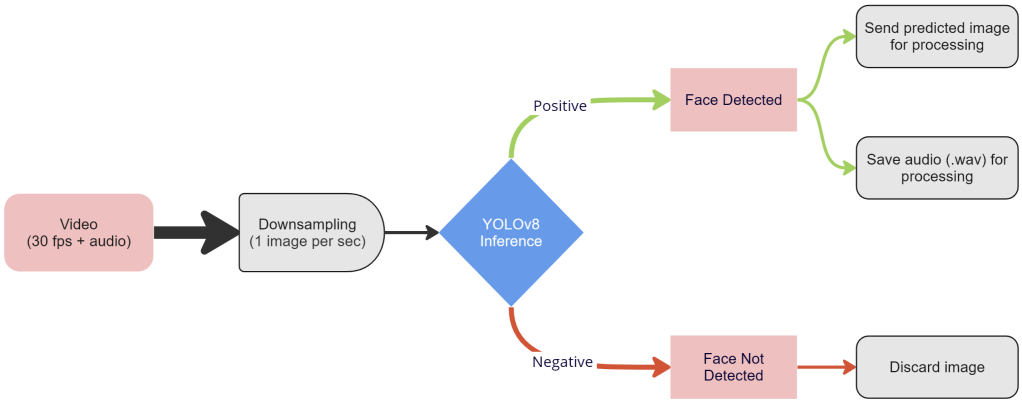


Figure 3: Object Detection Model Diagram

Given a video file, our script first downsamples the amount of images for processing. Instead of taking each image in a 30 frames per second (fps) video, one image is sampled every second and then processed.
When an image is selected, the YOLO model makes an inference on whether a face is present. If there is a positive prediction, the images are altered to generate better results for the visual sentiment model, cropped to only contain the bounding boxes, and stored under `output/<current_time>`. These images will later be used for visual sentiment analysis.
Additionally, when there are positive predictions, the object detection script also records the audio of the video, which is stored in `output/<current_time>/audio.wav` and sent to the audio sentiment model for processing.

### Understanding YOLOv8

YOLOv8 is an object detection algorithm that can detect any object with only a single forward pass through the neural network. We leveraged the YOLOv8 algorithm to perform facial detection for our model due to its high accuracy and versatility, especially for real-time inferences on live video feed.

- **High Performance**: YOLO achieves high accuracy by dividing the input image into a grid and simultaneously predicting bounding boxes and class probabilities for each grid cell.
- **Adaptability**: YOLO is adaptable to various object detection tasks and diverse datasets.
- **Real-Time**: YOLO's efficiency enables real-time object detection, suitable for applications requiring quick responses.

## Training

To perform the training, we utilized the public Roboflow software. We selected Roboflow because it allowed us to create the dataset rapidly by having a streamlined method to relabel the images. Additionally, Roboflow trains the models remotely on machines with better processing power which improves model accuracy and runtimes.
We trained the model numerous times, each time experimenting with different parameters such as varying image preprocessing steps and overlap bounds. Our final model downsamples the image definition, rotates and shears the image 30%, and applies grayscale to some images. This model performed the best by generating the most accurate bounding boxes. We imported this iteration from Roboflow and use it in our pipeline to make predictions on the video feed. You can find our training result from Roboflow.

**Training Graph for Final Model**

See Figure. 4

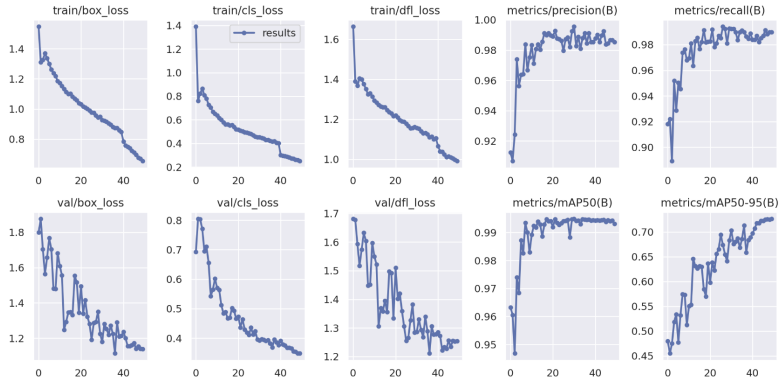


Figure 4: Object Detection Training Graph

## Results

When tested on our created dataset, our final model achieved all expected performance benchmarks:

- 96.3% mean average precision (mAP): defines the average "correct" percentage of the model's predicted bounding box compared with the ground truth bounding box
- 91.2% precision: measures the number of accurately predicted positive images over all positive predictions
- 91.8% recall: measures the proportion accurately predicted positives identified correctly

These results highlight the success of our facial detection algorithm, crucial for generating significant final results.

## Limitations and Future Work

- Variance in the size of the bounding box.
- Downsampling of images might miss potential faces.
- The algorithm might detect faces not relevant to the task.

Given more time, improvements would include better bounding boxes and a larger dataset more similar to the expected environments.

# Configuration

- `FPS`: Frames per second for segmenting human faces.
- `chunk`, `sample_format`, `channels`, `fs`, `seconds`: Audio recording settings.
- `stop_threshold`: Number of frames without face detection to stop recording.

**Note**:

- All input videos are assumed to be 30 fps.
- If a video file is provided as input, the script converts the entire video to an audio file.

# Visual Sentiment Analysis(Luyang Hu & Jinyi Wang)

## Dataset

The final dataset used for visual sentiment analysis is FER+(Face Expression Recognition Plus), an improvement over FER2013 with better quality labels obtained from crowd-sourcing.
Both datasets consist of 48x48 pixel grayscale images of faces, automatically registered to center the face in each image. The training set has 28,709 examples, and the test set has 3,589 examples.
The FER+ dataset has 8 labels, but for our project, we transformed it into binary labels (Positive and Negative) using a Python script `(scripts/fer_plus_data_trans.py)`. The processed dataset, `fer2013-ferplus.csv`, is available in the `dataset/ViT_training` folder.

## Model

### Legacy Model (CNN)

The initial model was a four-layer Convolutional Neural Network (CNN) with batch normalization and dropout layers to mitigate over-fitting. The model used an SGD optimizer and was trained on the FER+ dataset for 150 epochs. Training scripts can be found in `src/sentiment_analysis_visual/training_scripts/CNN_model.ipynb`.

### Final Model (ViT)

To further improve the performance, we later switched to the ViT (Visual Transformer) model, because of the following advantages:

- Robust to obstacles
- Transfer Learning

Here is the ViT architecture:

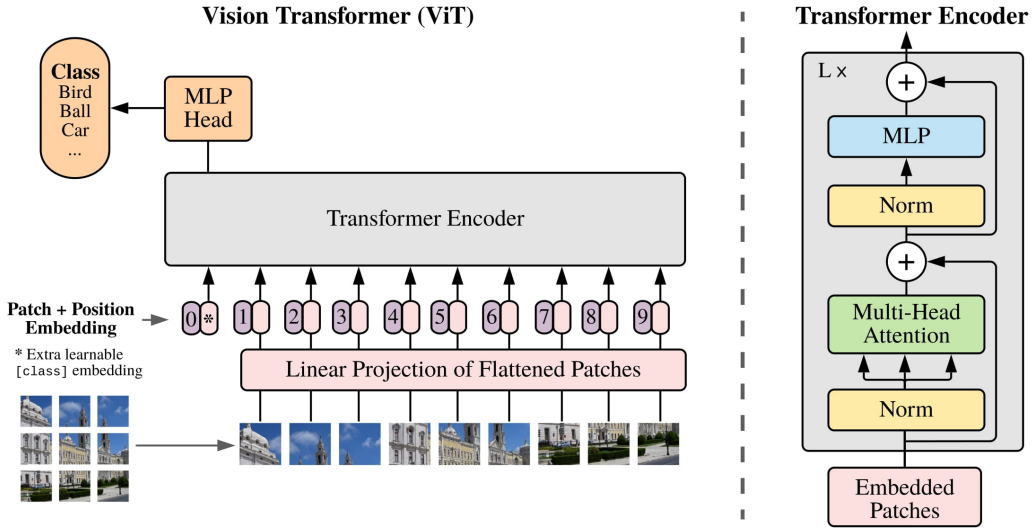


Figure 5: ViT Architecture

The input images are represented as a series of image patches, and ViT incorporates a self-attention mechanism as the original transformer model. Self-attention enables the models to capture long-range dependencies and relations between different parts of the image, which is beneficial to our image classification task.
Our model also takes advantage of transfer learning by pertaining to image-net21k and fine-tuned on our down-stream task, visual sentiment analysis. Here is the flow chart of our model in Figure. 6.

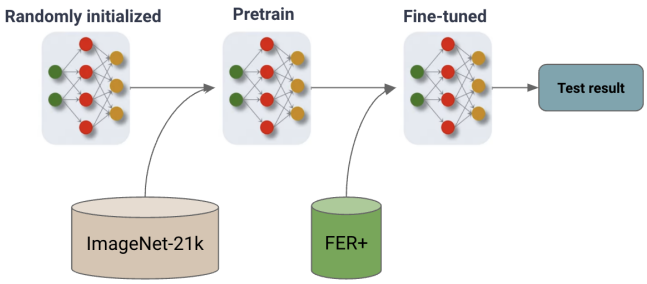


Figure 6: Transfer Learning Flow Chart

# Results

AUC value provides a single scalar value that summarizes the performance of a binary classification model across different decision thresholds. A higher AUC generally indicates a better-performing model. An AUC of suggests a model that performs no better than random chance, while an AUC of indicates a perfect classifier.

- The CNN-based model has achieved a test AUC of 0.76.
- The ViT model has achieved a test AUC of 0.937.

The following Figure. 7 shows a confusion matrix demonstrating the evaluation of the test set.

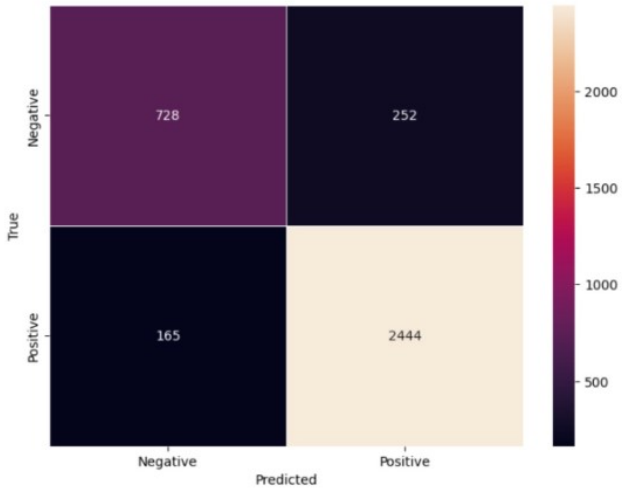


Figure 7: Confusion Matrix of ViT Model

## Preprocessing

Our preprocessing focuses more on matching the model dimensions and features, including:

- Resize the image to 48 * 48 pixels
- Convert it to grayscale
- Processed by the visual transformer feature extractor

## Feature: Differential Learning

Differential learning is designed to filter out the external emotions that are irrelevant to the parking experience. The initial design was to take separate video frames from the beginning and the end of the interaction and calculate the change in emotions. We elevated this idea by calculating emotions from all frames in the kiosk video and generating a trend for analysis.

Our current strategy:

- Calculate the regression line of that emotion trend
- If the regression line has a change(absolute difference between $y_{min}$ and $y_{max}$) less than 0.25, we consider there is no significant change in emotion scores. We will simply calculate the mean of all emotion scores and combine it with the audio model.
- Otherwise, we consider there is a change in emotion scores. Our current approach is naive, just setting the emotion score of negative change to 0.25 and positive change to 0.75. Those values are treated as parameters that can be changed according to testing results.

The bash script will also generate a visualization of emotions with the regression line, like Figure. 8:

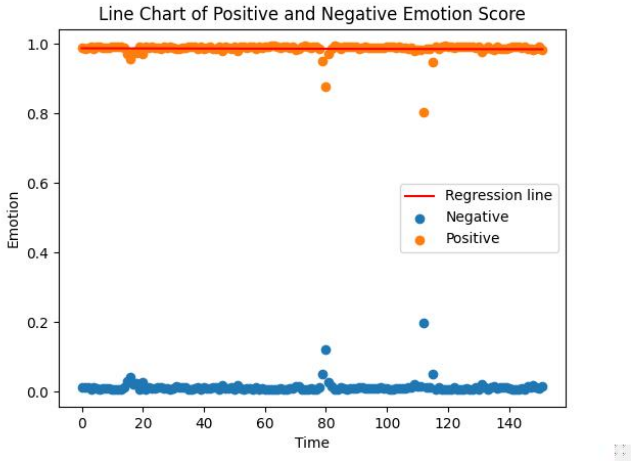


Figure 8: Visualization of Emotions with Regression Line

## Limitation and Future Work

Our model yielded a very high result on the public dataset, FER+. However, its ability to generate real data is unsatisfying. We believe this phenomenon not result from the model itself but the dataset. The FER+ dataset, as one of the most popular facial expression recognition datasets public online, is very outdated. The data format is too simple, 48 *48 grayscale, to handle our problem. Here are some examples in Figure. 9:

There is obviously much bad data that is too dark to be interpreted. Moreover, many facial expressions are very obvious and dramatic, making our model hard to learn to capture those fine-grained sentiments. Its low-resolution nature deteriorates this situation even more.

To address this problem, we tried to use transfer learning(mentioned above in the model section) and some level of data preprocessing. Nevertheless, the fundamental problem with the dataset cannot be solved. We also tried to attain the dataset by ourselves by taking videos mimicking the actual FLASH kiosk setting. This action was halted due to the restrictions of Human Subject Study protocols.

In the future, if FLASH can acquire the dataset from the actual environment and use those to train the model, it would be optimal. Alternatively, people can also try other more advanced datasets, such as RAF-DB. For future improvement on the dataset, we suggest considering using the RAF-DB dataset because it has color and contains more natural and real-world context for emotional expression.

Figure 9: Visualization of Emotions with Regression Line

# Audio Sentiment Analysis(Zhe Chen & Rui Wu)

## Dataset

For audio sentiment analysis, our primary dataset is the dair-ai/emotion dataset, which is a comprehensive collection of data specifically tailored for emotion recognition tasks. This dataset is available on the Hugging Face datasets platform and can be accessed here: dair-ai/emotion dataset. It is a dataset of English Twitter messages with six basic emotions: anger, fear, joy, love, sadness, and surprise, providing a robust foundation for training and evaluating our models.
As our project only targets positive and negative emotions, we used excel and GPT to help transform it into binary labels. Here are our reclassified emotions on the dataset:

- Positive: joy, love, surprise
- Negative: anger, fear, and sadness

You can attain our processed dataset, `train_reduced.csv` and `test_reduced.csv`, in the format of csv in the `dataset/LSTM_training folder`.

## Model

### Overall Structure

See Figure. 10
Given an audio file, we first utilize the SpeechRecognition library available at SpeechRecognition PyPI for converting audio files into transcripts. This library, with its high accuracy of 95.1%, effectively transcribes spoken words into text. The accuracy metric is sourced from a 2017 report by 9to5Google, highlighting the library's sophistication and reliability.
In the second stage of our audio sentiment analysis, we delve into analyzing the sentiment of the transcribed text. This is achieved through a model based on a **Bidirectional LSTM** (Long Short-Term Memory) network combined with an **Attention mechanism**.

### Understanding BiLSTM

- **Bidirectional LSTM Architecture**: Traditional LSTMs process data in a forward direction, which means they are well-suited to understanding context from the past. However, in sentiment analysis, understanding the context from both past and future is crucial. BiLSTM addresses this by processing data in both forward and backward directions (i.e., it looks at the sequence from start to end and end to start), effectively capturing context from both sides.
- **Contextual Understanding**: This bidirectional processing allows the model to have a broader understanding of the context in which words are used. For instance, the meaning and sentiment of a word can change significantly based on the words that precede or follow it.

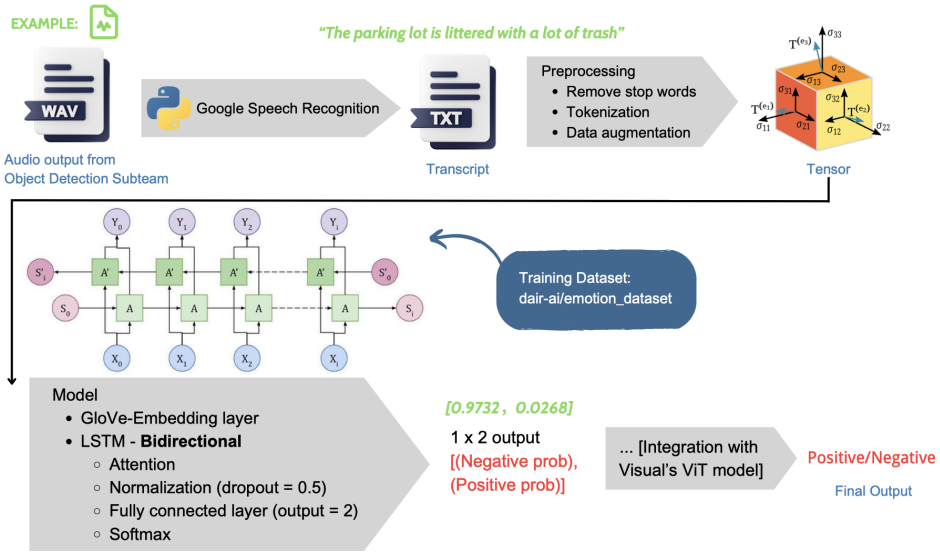


Figure 10: Audio Model Structure

### Attention Network

- **Selective Focus**: The Attention mechanism allows the model to focus on specific parts of the text that are more relevant to the sentiment being expressed, rather like how humans pay more attention to certain words or phrases when interpreting a sentence.
- **Enhanced Interpretation**: This results in a more nuanced interpretation of the text, as the model isn't constrained to treat all parts of the transcript equally. Instead, it learns to weigh different parts of the sentence to derive the overall sentiment.

## Training Process

See Figure. 11 for cross validation loss and Figure. 12 for validation accuracy.
The training process is illustrated through graphs showing cross-validation loss and validation accuracy. The training script is available in `src/sentiment_analysis_audio/training_scripts/train_audio.ipynb`.

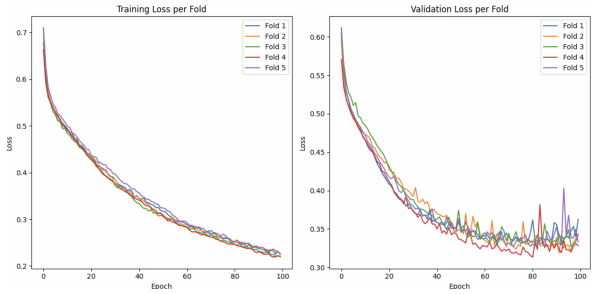


Figure 11: Cross Validation Loss

## Results

Our audio sentiment analysis model has achieved impressive results, including:

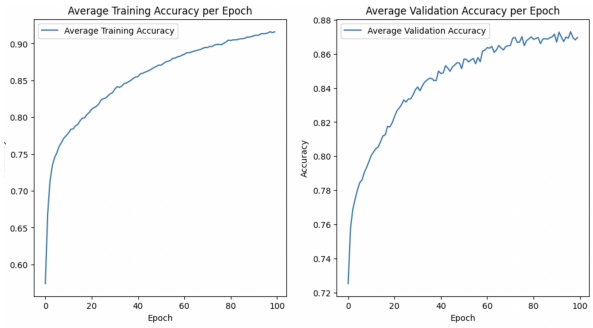


Figure 12: Validation Accuracy

- 0.94 AUROC (Area Under the Receiver Operating Characteristics): This metric indicates the model's ability to distinguish between different emotion classes effectively.
- 89.2% Accuracy: Demonstrating the model's overall effectiveness in correctly identifying the sentiments from audio transcripts.

These results validate the effectiveness of our approach in accurately analyzing sentiments from audio data, making it a valuable tool for various applications, including customer service analysis, psychological research, and more.

## Limitations and Future Work

Our project, while yielding promising results in training phase, has encountered several limitations that need to be acknowledged and addressed:

- **Handling High-Noise Environments** A significant limitation encountered in our project is the performance of the Google SpeechRecognition package under high-noise conditions. When the background noise is substantial, the package struggles to accurately transcribe audio files, often resulting in an `UnknownValueError`, thus failing the process of sentiment analysis. This error indicates the inability of the speech recognition technology to detect and process speech in these conditions.
- **Dataset Limitations** The dair-ai/emotion dataset, primarily consisting of Twitter messages, may not encompass the full spectrum of emotional expressions found in natural speech. Additionally, it is confined to English language and basic emotions, which might limit the model's applicability to a more diverse linguistic and emotional range.
- **Contextual Nuances** While the BiLSTM with Attention mechanism is adept at capturing context, there might be challenges in accurately interpreting subtleties such as sarcasm, irony, or cultural nuances in speech.
- **Generalization to Real-World Data** The model's performance on controlled datasets might not fully reflect its effectiveness in real-world scenarios, where audio quality, background noise, and varied speech patterns play a significant role.

To overcome these limitations and enhance our model, we propose several areas for future work:

- **Enhanced Speech Recognition** Integrating advanced speech recognition technologies, especially those capable of handling diverse accents and dialects (such as Alibaba Cloud Intelligent Speech Interaction), would improve the accuracy of our transcriptions, thereby enhancing sentiment analysis.
- **Expanding the Dataset** Incorporating a more diverse range of datasets, including different languages and more varied emotional expressions, will help in developing a more robust and universally applicable model.

- **Real-World Testing and Adaptation** Testing the model in real-world scenarios and adapting it based on the feedback and results obtained will be crucial for its practical applicability.